

Applied Data Analysis (CS401)



Lecture 11 Handling text data Part II 26 Nov 2025

Maria Brbic

EPFL

1

Announcements



- To all ADAmericans:
Happy Thanksgiving!
- Homework H2 due this **today** EOD
 - Reminder: We won't answer questions asked during final 24h
- Projects:
 - Milestone P2 grades have been released
 - **Project Milestone P3 released today!**
- Friday's lab session:
 - Exercises on handling text ([Exercise 10](#))

2

Feedback

Give us feedback on this lecture here:

<https://go.epfl.ch/ada2025-lec11-feedback>

- What did you (not) like about this lecture?
- What was (not) well explained?
- On what would you like more (fewer) details?
- ...

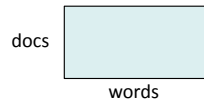
3

Recap

Let me open my **bag of tricks for bags of words** for you! But only if you were good children...



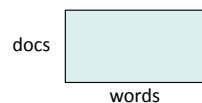
Reminder:
bag-of-words matrix



4

Revisiting the 4 typical tasks

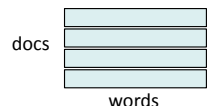
- Document retrieval
- Document classification
- Sentiment analysis
- Topic detection
- TF-IDF matrix to the rescue
 - Entry for doc d , word w :
 $tf(d, w) * idf(w)$



5

Typical task 1: document retrieval

- Nearest-neighbor method in spirit of kNN
- Compare query doc q to all documents in the collection (i.e., rows of the TF-IDF matrix)
- Rank docs from collection in increasing order of distance
- Distance metrics
 - Typically cosine distance ($= 1 - \text{cosine similarity}$)
 - Recall: cosine similarity of q and $v = \langle q/|q|, v/|v| \rangle$
 - If rows are L2-normalized, may simply take dot products $\langle q, v \rangle$



6

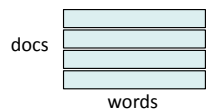
Typical task 1: document retrieval

- This is just the most basic approach
- For efficiency
 - Start by filtering documents by presence of query terms (use efficient full-text index)
 - Hugely narrows down set of documents to be ranked
- Google et al. do much more...
 - Query-independent relevance: PageRank
 - Boost recent results
 - Personalization, contextualization
 - ...

7

Typical task 2: document classification

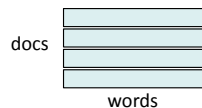
- Use TF-IDF matrix as feature matrix for supervised methods (cf. lecture 7)
- Often more features (words) than documents
- What's the danger with this?
- High model capacity can lead to overfitting (high variance)
- Potential solutions:
 - Use more data (i.e., more labeled training docs)
 - Decrease model capacity:
 - Feature selection
 - Regularization (two slides from now)
 - Dimensionality reduction (a few slides from now)
 - Use ensemble methods such as random forests



8

Typical task 3: sentiment analysis

- When treated as classification: Ctrl-C Ctrl-V previous slide
- When treated as regression: Pretty much the same (most supervised methods work for both classification and regression)



9

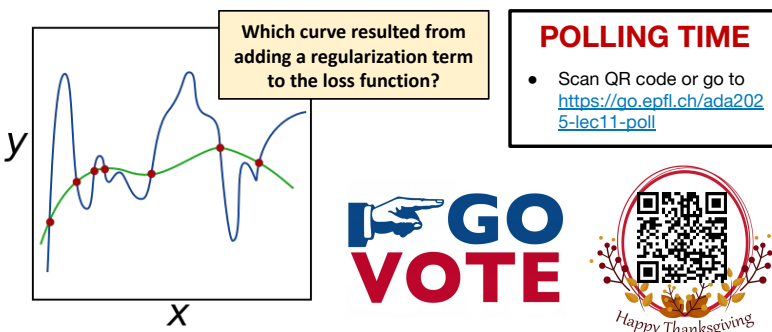
Regularization

- E.g., linear regression: Find weight vector β that minimizes $\sum_{i=1}^n (y_i - \mathbf{x}_i^\top \beta)^2$
- ($\mathbf{x}_i$: feature vector of i -th data point; y_i : label of i -th data point, e.g., sentiment [1–5 stars] expressed in document i)
- If one word j appears only in docs with sentiment 5, we can obtain very small training error on these docs by making β_j large enough
- But doesn't generalize to unseen test data!
- Remedy: penalize very large positive and very large negative weights:

$$\text{minimize } \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

10

Regularization

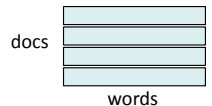


POLLING TIME

- Scan QR code or go to <https://go.epfl.ch/ada2025-lec11-poll>

Typical task 4: topic detection

- Cluster rows of TF-IDF matrix (each row a data point)
- Manually inspect clusters and label them with descriptive names (e.g., "news", "sports", "romance", "tech", "politics")
- In principle, may use k-means, k-medoids, etc.
- But can be difficult if dimensionality is large (#words $\gg$ #docs)
 - "Curse of dimensionality"
 - Many outliers



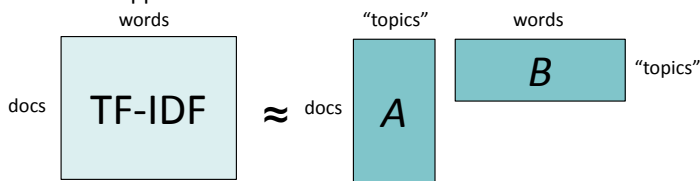
What else can we do?



12

Typical task 4: topic detection

- Alternative approach: **matrix factorization**



- Assume docs and words have representation in (latent) “topic space”
- (IDF-weighted) word frequency modeled as dot product of doc’s vectors and word’s vectors in topic space
- #topics $\ll$ #words ($\rightarrow$ “dimensionality reduction”): $D \times W \rightarrow (D+W) \times T$
- Topics interpretable in doc space (A ’s cols) and word space (B ’s rows) ¹³

Typical task 4: topic detection

- Optimization problem:

- Find A, B such that AB is as close to TF-IDF matrix as possible

- That is, minimize $\sum_{d=1}^N \sum_{w=1}^M (T_{dw} - A_d B_w)^2$

where T is TF-IDF matrix, A_d is d -th row of A , and B_w is w -th column of B

- This is called **latent semantic analysis (LSA)**

14

Typical task 4: topic detection

You already know how to efficiently compute this, from your linear algebra class: **singular-value decomposition** (SVD)



- $T = USV^T$
- Freebie: columns of U and V are orthonormal bases (yay!)
- S is diagonal (with values in decreasing order) and captures “importance” of topic (amount of variation in corpus w.r.t. topic)
- If you want k topics, keep only the first k columns of U and V , and the first k rows and columns of S
 $\rightarrow U', S', V'$
- E.g., $A = U', B = S'V'^T$ or $A = U'S', B = V'^T$

15

Typical task 4: topic detection

- Recall potential problem with clustering and classification and regression: “curse of dimensionality”
- Matrix factorization via LSA solves these problems for you:
 - Use A instead of original TF-IDF matrix
 - That is, cluster (or learn to classify or regress) in topic space, rather than word space

- Topic representation from LSA is simply a vector, not a probability distribution over topics
- Probabilistic: LDA = Latent Dirichlet Allocation (p.t.o.)

16

Commercial break



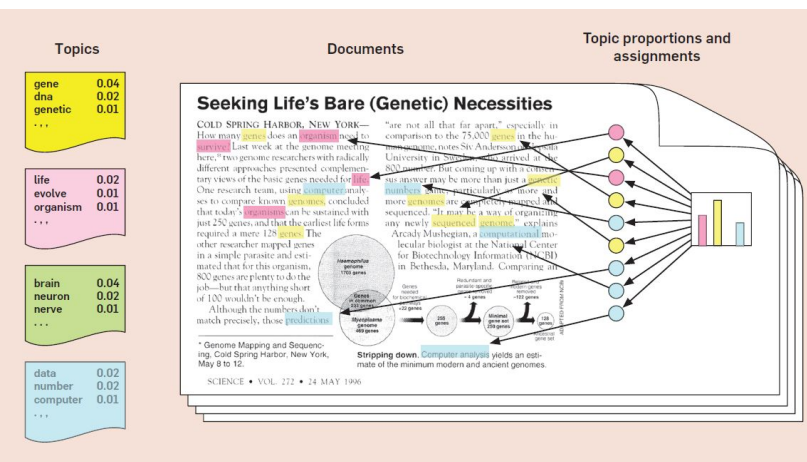
ada 
ça roule pour moi

17

LDA: probabilistic topic modeling

- Latent Dirichlet Allocation** (*not* Latent Discriminant Analysis!)
- Document := bag of words
- Topic := probability distribution over words
- Each document has a (latent) distribution over topics
- “Generative story” for generating a doc of length n :
 - d := sample a topic distribution for the doc ($\leftarrow$ “Dirichlet”)
 - for $i = 1, \dots, n$
 - t := sample a topic from topic distribution d
 - w := sample a word from topic t
 - Add w to the bag of words of the doc to be generated

18



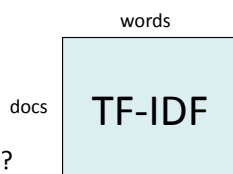
Topic inference in LDA

- LDA is unsupervised (topics come out “magically”)
- Input:
 - Docs represented as bags of words
 - Number K of topics
- Output:
 - K topics (distributions over words)
 - For each doc: distribution over K topics
- How is this done?
 - Find distributions (i.e., topics, docs) that maximize the likelihood of the observed documents (maximum likelihood)

20

Question:

- “Which of these word pairs is more closely related?”
 - (car, bus)
 - (car, astronaut)
- How to quantify this?
- Detour:
 - How to quantify closeness of two docs?
 - E.g., cosine of **rows** of TF-IDF matrix
- Retour:
 - How to quantify closeness of two words?
 - E.g., cosine of **cols** of TF-IDF matrix



21

Sparsity in TF-IDF matrix

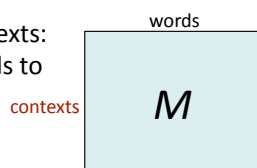
- Two docs (i.e., rows of TF-IDF matrix)
 - “Do you love men?”
 - “Adorest thou the likes of Adam?”
 - Cosine of row vectors of TF-IDF matrix == 0
- Same problem when comparing two words (i.e., cols of matrix)
- Solution:
 - Move from sparse to dense vectors
 - But how?
 - Latent semantic analysis (LSA)!
 - Use columns of B as dense vectors representing words



22

“Word vectors”

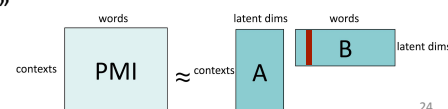
- Columns of TF-IDF matrix (sparse) or of word-by-topic matrix B (dense)
- Problem:
 - Entire doc treated as one bag of words
 - All information about word proximity, syntax, etc., is lost
- Solution:
 - Instead of full docs, consider local contexts: windows of L (e.g., 3) consecutive words to **left and right** of the **target word**
 - Rows of matrix: not docs, but contexts



“Word vectors”

- What to use as entries of word/context matrix?
- Straightforward: same as TF-IDF, but with contexts as “pseudo-docs”: $M[c, w] = \text{TF-IDF}(c, w)$
- May use any other measures of statistical association
- E.g., pointwise mutual information (PMI):

$$M[c, w] = \text{PMI}(c, w) = \log \frac{\Pr(c, w)}{\Pr(c) \Pr(w)}$$
 “How much more likely are c and w to occur together than if they were independent?”
- word2vec: factor PMI matrix and use columns of B as word vectors



24

Beyond bags of words

25

From words to texts

- Word vectors represent, well, words
- How to represent larger units, such as sentences, paragraphs, docs?
- Typical approach: take sum/average of word vectors
- Note: this is roughly also what bags of words are (when using “one-hot” encoding for words, i.e., vector with exactly one 1, rest 0)
- More recently: **learn** vectors for longer units
 - [Cr5](#), [sent2vec](#)
 - Convolutional neural networks
 - Recurrent neural networks, e.g., LSTM, [ELMo](#)
 - Transformer-based models, e.g., [BERT](#) (next slides), GPT-*

26

Contextualized word vectors

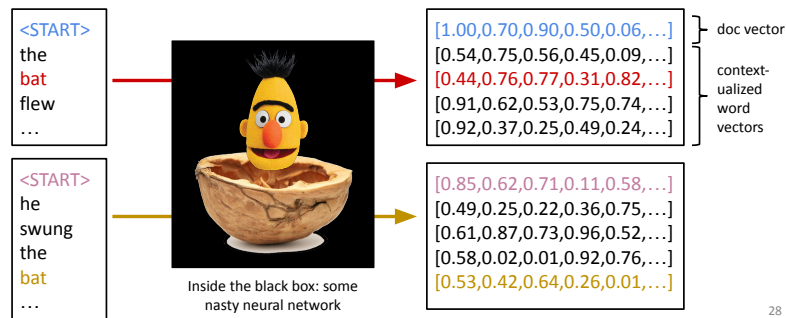
- Motivating example:
 - “The **bat** flew into the cave.”
 - vs. “He swung the **bat** and hit a home run.”
- Classic word vectors (e.g., word2vec) cannot distinguish these two cases; same vector used for both instances of “bat”
- Solution: contextualized word vectors
 - E.g., BERT



27

BERT in a nutshell

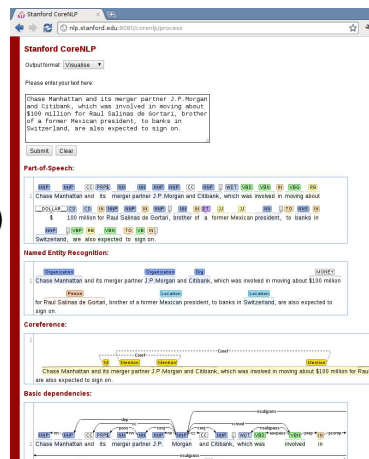
- [Introduced](#) in 2018 by Google Research



28

NLP pipeline

- Tokenization
- Sentence splitting
- Part-of-speech (POS) tagging
- Named-entity recognition (NER)
- Coreference resolution
- Parsing
 - Shallow parsing (a.k.a. chunking)
 - Constituency parsing
 - Dependency parsing

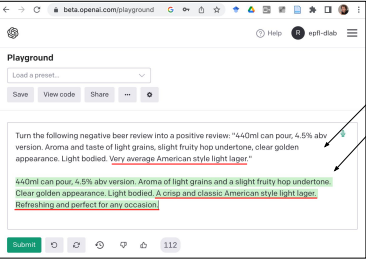


NLP pipeline

- Implemented by [Stanford CoreNLP](#), [nltk](#), [spaCy](#), etc.
- Sequential model
 - Fixed order of steps
 - Early errors will propagate downstream
 - Fixed order not optimal for all cases (e.g., syntax usually done before semantics, but semantics might be useful for inferring syntax)
- Hence, current research: learn all tasks jointly ([early example](#))
- To learn how all this magic is implemented
 - Take [CS-431](#) (Intro to NLP), [CS-552](#) (Modern NLP)

30

Today’s trend: generative language models



- E.g., OpenAI GPT-*, ChatGPT
 - Input: text
 - Output: text
 - Many NLP tasks can be formulated in this framework, by “prompting” the language model with the right input
- 
- The diagram shows a tree with a brown trunk and many colorful, circular leaves. Each leaf contains a text-based NLP task. The tasks are grouped into several categories represented by different colors: yellow for reasoning tasks (Question Answering, Semantic Parsing, Logical Inference, Common-Sense Reasoning, Pattern Recognition), purple for translation and dialogue (Translation, Dialogue, Joke Explanations, Physics QA), green for general knowledge and summarization (General Knowledge, Summarization, Reading Comprehension), blue for arithmetic and code completion (Arithmetic, Code Completion), and red for language understanding (Language Understanding).

Feedback

Give us feedback on this lecture here:
<https://go.epfl.ch/ada2025-lec11-feedback>

- What did you (not) like about this lecture?
- What was (not) well explained?
- On what would you like more (fewer) details?
- ...